Written By Michael Pogrebinsky

How to Create Software Architecture Diagrams with Code - The Software Architects' Toolbox

Software architecture diagrams are essential for visualizing systems and communicating the software architecture to stakeholders. Traditionally, software architecture diagrams were created manually, which was tedious and time-consuming. Updating those diagrams was also very hard, making them go out of date quickly and reducing their usefulness.

The "diagrams-as-code" approach transforms this process by allowing software architects to describe software architecture components and their relationships through code. Using code to describe software architecture diagrams enables tracking changes with version control and seamless integration into development workflows. Also, making and reviewing changes is very easy, so software architecture diagrams are always up-to-date and reflect the system's current state.

Top Tools for Software Architecture Diagrams with Code

There is a variety of tools available for creating software architecture diagrams with code, each offering unique features and benefits. Here are some of the most popular options that I've used myself as a software architect for creating software architecture diagrams. Those tools have minimal overlap, and you will likely use all 3 at one point or another as a professional software architect.

1. PlantUML

PlantUML is a versatile tool that supports the most common UML (Unified Modelling Langauge) diagrams, such as Sequence diagrams, Use case diagrams, Class diagrams, Object diagrams, Activity diagrams, Component diagrams, Deployment diagrams, State diagrams, and Timing diagrams.

317
Shares

In addition, it supports Non-UML diagrams to describe:

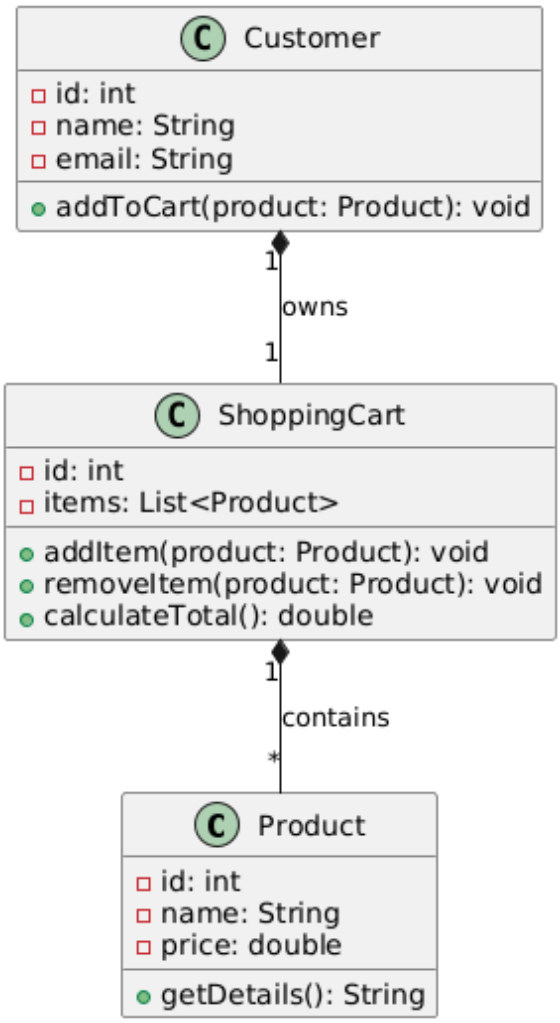
- Complex data types like JSON and YAML.
- Enterprise Architecture communication channels between stakeholders using ArchiMate diagrams
- Entity relationship diagrams are used to model data for databases, etc.

And so on.

Example 1: Creating a UML Class Diagram with PlantUML

Here is a simple example of a UML class diagram defined in PlantUML, and the visual architecture diagram is automatically created from this code.

```
@startuml
class Product {
  - id: int
  - name: String
  - price: double
  + getDetails(): String
}

class Customer {
  - id: int
  - name: String
  - email: String
  + addToCart(product: Product):
void
}

class ShoppingCart {
  - id: int
  - items: List<Product>
  + addItem(product: Product):
void
  + removeItem(product: Product):
void
  + calculateTotal(): double
}

Customer "1" *-- "1" ShoppingCart
: owns
ShoppingCart "1" *-- "*" Product :
contains
@enduml
```



317
Shares

As you can see in the code above, all the classes are intuitively described using the *class* keyword. In each class, all the attributes and methods of the class are declared using a -/+ indicating their access control (private/public).
At the bottom of the code snippet, we describe the relationship between the classes or their instances. The relationships PlantUML supports are association, inheritance, implementation, dependency, aggregation, and composition.

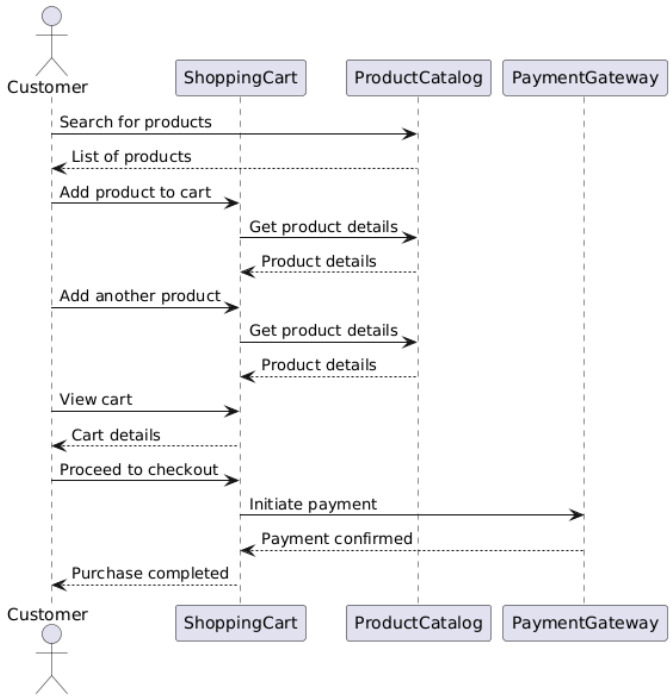
For example:

```
Customer "1" *-- "1" ShoppingCart : owns
```

creates a composition relationship where each customer has only one shopping cart, and each shopping cart can belong to only one customer.

Example 2: Creating a UML Sequence Diagram With PlantUML

```
@startuml
actor Customer
participant "ShoppingCart" as Cart
participant "ProductCatalog" as
Catalog
participant "PaymentGateway" as
Payment

Customer -> Catalog: Search for
products
Catalog --> Customer: List of
products

Customer -> Cart: Add product to
cart
Cart -> Catalog: Get product
details
Catalog --> Cart: Product details

Customer -> Cart: Add another
product
Cart -> Catalog: Get product
details
Catalog --> Cart: Product details

Customer -> Cart: View cart
Cart --> Customer: Cart details
```



317
Shares

```
checkout
Cart -> Payment: Initiate payment
Payment --> Cart: Payment
confirmed

Cart --> Customer: Purchase
completed
@enduml
```

Since a sequence diagram is a behavioral diagram, we describe only the relationships between the actors/objects in the system, which can either be humans or system components like objects, applications, etc. The interactions are described using arrows "—>" from one actor to another actor, with a description of the action after the colon symbol.

Pros of PlantUML

PlantUML supports many software architecture diagraming notations, including UML — the standard modeling language for structural and behavioral diagrams in software design. It's easy to use and has a fairly low learning curve. You can also customize the visual aspect of your diagrams using per-components styles and global themes.

Cons of PlantUML

While it's fairly easy to customize with color themes, its layout is a bit harder to control. It is also mainly a diagramming tool and not a modeling tool. In other words, if you want to describe the same system components but use multiple diagrams, any change will have to be done in each diagram separately to keep all the diagrams consistent.
Another big drawback is that it is unsuitable for creating software architecture diagrams for large-scale systems or cloud architecture diagrams.

2. Diagrams as Code (Python Library)

Diagrams or **Diagrams as Code** lets you create **cloud system diagrams** using standard **Python** code. It is a perfect tool to prototype new cloud-based systems or document existing software architectures in your company. It is also possible (though not supported out-of-the-box) to integrate it with "Infrastructure as a Code" tools to both control actual cloud infrastructure or reflect existing cloud infrastructure.

317
Shares

This software architecture diagraming tool supports many standard symbols from cloud vendors out-of-the-box, such as AWS, Azure, GCP, Alibaba Cloud, Oracle Cloud, IBM, On-Premise, etc.

It also supports many:

- **Open-source** and **SaaS** solutions like Kubernetes, Docker, Kafka, etc.
- **Monitoring tools** like Datadog, New Relic, PagerDuty, etc
- **Databases** like PostgreSQL, MySQL, MongoDB, etc.
- **Frameworks** like Angular, Flutter, NextJS, Spring Boot, Vue.js, etc.
- **Programming languages** like Java, C++, Javascript, Python, etc.

And much, much more.

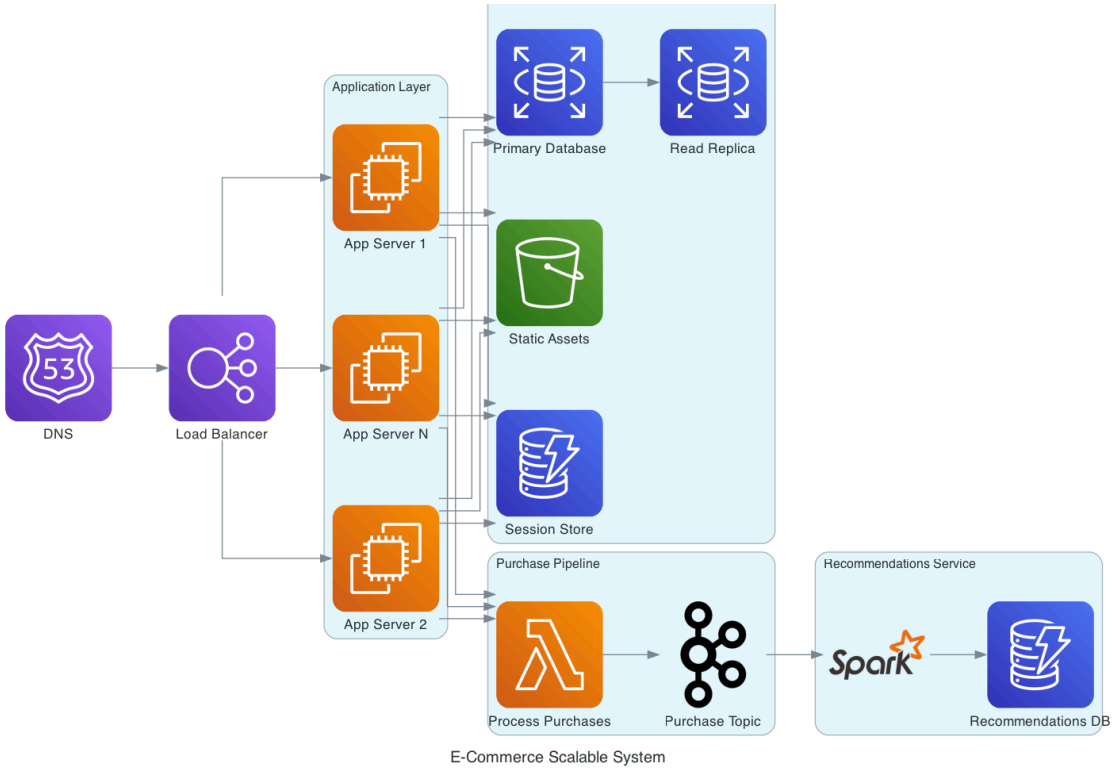
Example: Software Architecture Diagram of Fictious E-Commerce System on AWS

```
from diagrams import Diagram, Cluster
from diagrams.aws.compute import EC2, Lambda
from diagrams.aws.database import RDS, Dynamodb
from diagrams.aws.network import ELB, Route53
from diagrams.aws.storage import S3
from diagrams.onprem.queue import Kafka
from diagrams.onprem.analytics import Spark
from diagrams.aws.integration import SNS

with Diagram("E-Commerce Scalable System", show=False):
    # Route53 and Load Balancer
    dns = Route53("DNS")
    lb = ELB("Load Balancer")

    dns >> lb

    # Application Cluster
    with Cluster("Application Layer"):
        app_servers = [EC2("App Server 1"), EC2("App Server 2"), EC2("App
Server N")]
        lb >> app_servers

    # Storage and Database
    with Cluster("Storage and Databases"):
        db = RDS("Primary Database")
        db_replica = RDS("Read Replica")
        db >> db_replica
        cache = Dynamodb("Session Store")
        static_assets = S3("Static Assets")
```

317
Shares

```
    app_servers >> cache
    app_servers >> static_assets

    # Purchase Processing
    with Cluster("Purchase Pipeline"):
        purchase_lambda = Lambda("Process Purchases")
        kafka_topic = Kafka("Purchase Topic")
        purchase_lambda >> kafka_topic

    app_servers >> purchase_lambda

    # Recommendations System
    with Cluster("Recommendations Service"):
        recommender = Spark()
        recommender_db = Dynamodb("Recommendations DB")
        kafka_topic >> recommender >> recommender_db
```

This example includes AWS services like Route 53, ELB, EC2, S3, RDS, DynamoDB, and Lambda, as well as open-source technologies like Apache Spark and Kafka. The code is also very intuitive and easy to understand since it uses Python, a very popular programming and scripting language. All you need to do is import the relevant classes from the *diagrams* module. The generated diagram uses an automatic layout and modern colors without any special code.



This tool uses standard Python, so learning a new diagramming language is unnecessary if you already know Python. You can easily spot errors through an IDE like PyCharm or get compilation errors while running the tool. The library is also very intuitive, and the generated diagrams appear in a visually pleasing, professional, and easy-to-read format. This software architecture diagraming library can also be easily extended with custom symbols if required, even though it's rarely necessary since all major systems are supported out of the box.

Cons of Diagram as Code with Python

Compared to PlantUML, there's no support for UML or any other software architecture diagraming notations besides cloud infrastructure. Though it does have support for C4 Modeling, the support is very minimal. There's also very little customization available in terms of the visual aspects of the diagram, such as styles, colors, and layout. Creating animations to visualize data flow between different components is also impossible.

Want to Become a Software Architect?

Becoming a **software architect** and **technology leader** is the ultimate goal for every software engineer. But you don't need to wait for it to happen sometime in the far future!

In this guide, I share with you **the 5 proven steps** to becoming a software architect and technology leader today.

Use this free PDF guide to pave your path to success. Your biggest career breakthrough as an engineer is closer than you think.

Get Your Free E-Book Now!

First Name

Email Address

Download Now

3. Structurizr

Unlike PlantUML and Diagrams, which are diagramming tools, Structurizr is a modeling tool specifically designed for the C4 model for visualizing software architecture. C4, created by Simon Brown, is a standardized model to visualize software architecture using 4 views:

- *Context View*
- *Container View*
- *Component View*
- *Code View*.

Those 4 views allow us to represent the system on 4 different levels of abstraction or granularity.

Other diagramming tools are not suitable for creating software architecture diagrams for the C4 model because changes in diagrams in one view are not reflected automatically in other views where the same element/component is described.
Structurizr is the only tool that is aware of that connection between elements on different views, which keeps all the software architecture diagrams in sync with each other at all times.

Example: C4 Model of a Rideshare System

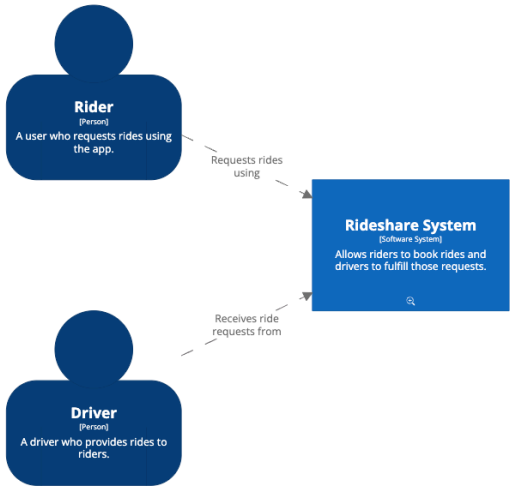
```
workspace {
    model {
        user = person "Rider" "A user who requests rides using the app."
        driver = person "Driver" "A driver who provides rides to riders."

        rideshareSystem = softwareSystem "Rideshare System" "Allows riders to
book rides and drivers to fulfill those requests." {
```

317
Shares

```
            webAppService = container "Web Application Backend" "Handles
webpage requests and user logins" "Java SpringBoot"
            mobileApp = container "Mobile Application" "Allows drivers to
accept and manage ride requests." "Flutter"
            apiGateway = container "API" "Handles requests from the web and
mobile applications." "AWS API Gateway"
            database = container "Database" "Stores user, driver, and ride
information." "PostgreSQL" {
                tags "Database"
            }
            paymentService = container "Payment Service" "Processes ride
payments." "Stripe"
            recommendationEngine = container "Recommendation Engine"
"Suggests ride matches and optimizes routes." "Python"

            webAppFrontend -> apiGateway "Makes API calls to"
            apiGateway -> webAppService "Routes API calls to"
            mobileApp -> apiGateway "Makes API calls to"
            webAppService -> database "Reads from and writes to"
            apiGateway -> paymentService "Processes payments via"
            apiGateway -> recommendationEngine "Fetches recommendations from"
        }

        user -> webAppFrontend "Requests rides using"
        driver -> webAppFrontend "Receives ride requests from"
        user -> mobileApp "Requests rides using"
        driver -> mobileApp "Receives ride requests from"
    }

    views {
        systemContext rideshareSystem {
            include *
            autolayout lr
        }

        container rideshareSystem {
            include *
            autolayout lr
        }

        styles {
            element "Container" {
                background #438dd5
                color #ffffff
            }

            element "Person" {
                shape person
                background #08427b
                color #ffffff
            }
```

```
                background #1168bd
                color #ffffff
            }
            element "Database" {
                background orange
                color black
                shape cylinder
            }
        }
    }
}
```

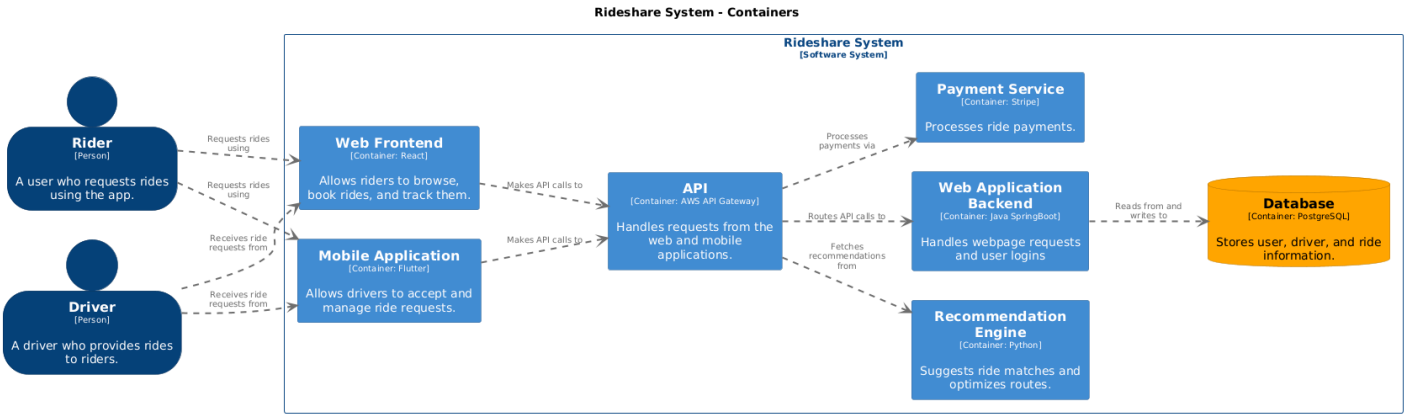
At first glance, it may seem a bit more complex than any of the diagrammatic tools we have seen so far. However, notice that we don't just show one diagram here. We describe a **model** of our entire system, with 2 views (Context View and a Container View).
Looking at the *Context View*, the diagram generated by Structurizr will look like this:



[System Context] Rideshare System

However, if we click on the Rideshare System (the system we are designing), we will "zoom in" to the *Container View*, which shows us all the containers (applications and databases) of our system.

317
Shares



Now, if we want to show the components of one of the containers, such as the Web Application Backend, we can take this a step further and add a *Component View* for this container.

To do that, we replace the component definition of the Web Application Service:

```
webAppService = container "Web Application Backend" "Handles webpage requests
and user logins" "Java SpringBoot"
```

with the definition that describes its internal components and their relationships:

```
webAppService = container "Web Application Backend" "Handles webpage requests
and user logins" "Java SpringBoot" {
    restController = component "REST Controller" "Handles HTTP requests from
the frontend"
    usersService = component "Users Controller" "Handles Business Logic for
Riders and Drivers"
    usersRepository = component "ORM" "Translates operations on Business
Entities to Database Operations"

    restController -> usersService "Parses Request to passes to the service
for processing"
    usersService -> usersRepository "Applies business logic and passes to the
ORM for communication with the database"
}
```

Now that we have defined the internal structure of the Web Application, we need to add the relationship between the external API Gateway container to the REST Controller component inside the Web Application Backend and the relationship between the internal UsersRepository component of the Web Application Backend to the external Database container:

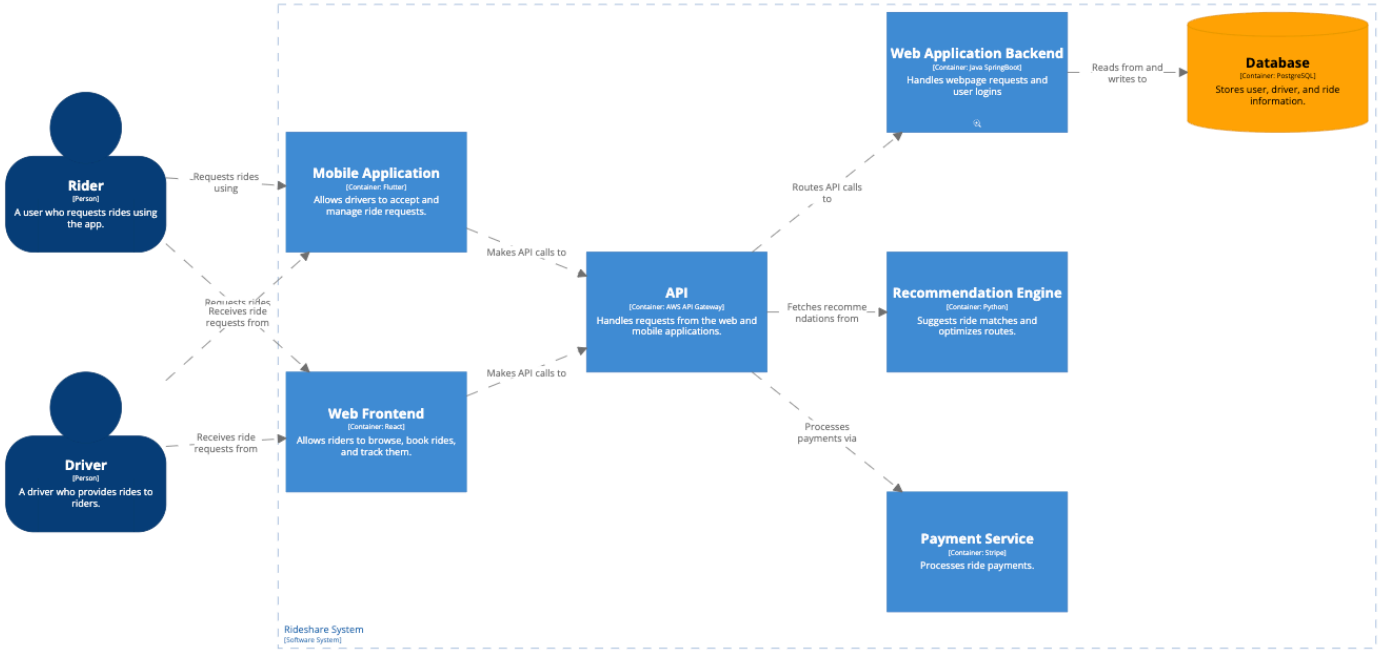
```
usersRepository -> database
```

Finally, inside the *views* block, we need to define the component view for the web application container:

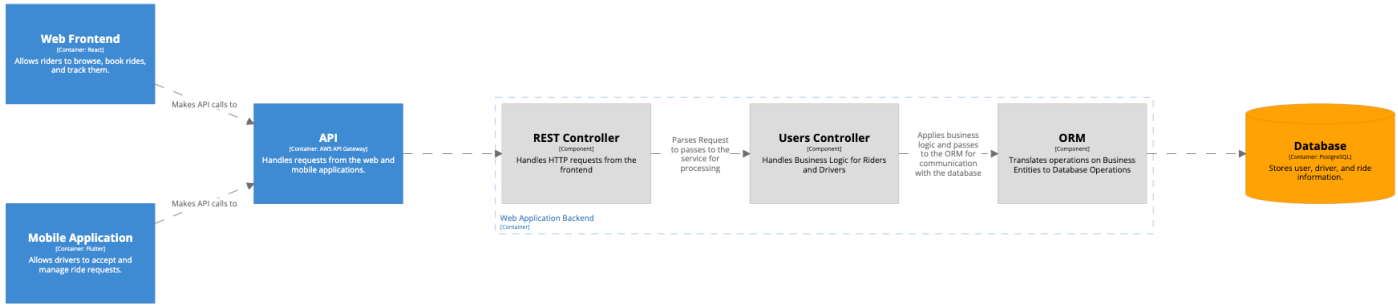
```
component webAppService {
    include *
    include mobileApp webAppFrontend
    autolayout lr
}
```

And that's it!

Now, the Web Application Container has a "zoom-in" icon which allows us to view its internal components:



If we want to zoom in and explore the internal structure of the Web Application Backend container, we can click on it and observe its Component View:



So, as we can see, using the C4 Model and the Structurizr DSL, we can create multiple diagrams for the same system, which all stay in sync if we make any changes.

Become a Software Architect: Take the Next Step in Your Career

Creating software architecture diagrams is just one part of what it takes to become a software architect. If you're excited about designing systems, leading teams, and shaping the technical direction of projects, you're already on the path to this rewarding career.

But how do you get there?

I've put together a free guide: **"The 5 Proven Steps to Becoming a Software Architect and Technology Leader,"** to help software engineers like you take the next step. Inside, you'll find actionable tips on building the skills, experience, and mindset you need to transition into this high-impact role of a software architect.

Get your Free Guide to Becoming a Software Architect

Becoming a **software architect** and **technology leader** is the ultimate goal for every software engineer. But you don't need to wait for it to happen sometime in the far future!

In this guide, I share with you **the 5 proven steps** to becoming a software architect and technology leader today.

317
Shares